Performance Analysis – Parallel Matrix Multiplication

Ehtesham Faraz

February 19, 2026

Analysis of row_wise_matrix_mult.c

The provided `row_wise_matrix_mult.c` implementation uses a **Master-Worker** pattern with dynamic load balancing. This pattern could be useful for tasks with inconsistent workloads, but it introduces major bottlenecks for matrix multiplication.

Communication Bottlenecks: The most critical performance flaw is the broadcast strategy. Lines 113-116 and 162-165 showed that `matrix_b` is broadcasted **row-by-row** inside a loop (`MPI_Bcast` called N times). This causes a massive latency overhead due to network handshakes for every single row, rather than streaming the data in a single large message. Additionally, the master process (Rank 0) acts as a central bottleneck. It must individually send rows of Matrix A to workers and receive results one by one. As the number of processes (P) increases, Rank 0 becomes overwhelmed, leading to poor scalability.

Memory and Initialization: The initialization is strictly serial. Rank 0 allocates and populates the entirety of matrices A, B, and C (approximately 1.5 GB for $N = 8000$). This limits the problem size to the RAM available on a single node.

EduMPI observation of row_wise_matrix_mult.c

Communication Bottlenecks (The "Red" State): Figure 1 shows the execution state of the unoptimized code. The abundance of **red cubes** in the EduMPI visualization indicates that processes are spending a majority of their time in communication or waiting states rather than computing. This is caused by the loop-based broadcast (Lines 113-116 of the source code), where the Master node (Rank 0) broadcasts `matrix_b` one row at a time. This created a serialization point where workers must wait for the network to deliver data row-by-row before they can proceed.

Visual Topology: The visualization shows a "Star" pattern where traffic is heavily centralized around Rank 0. As the process count increases, the Master node becomes overwhelmed, leading to the high waiting times shown in red.

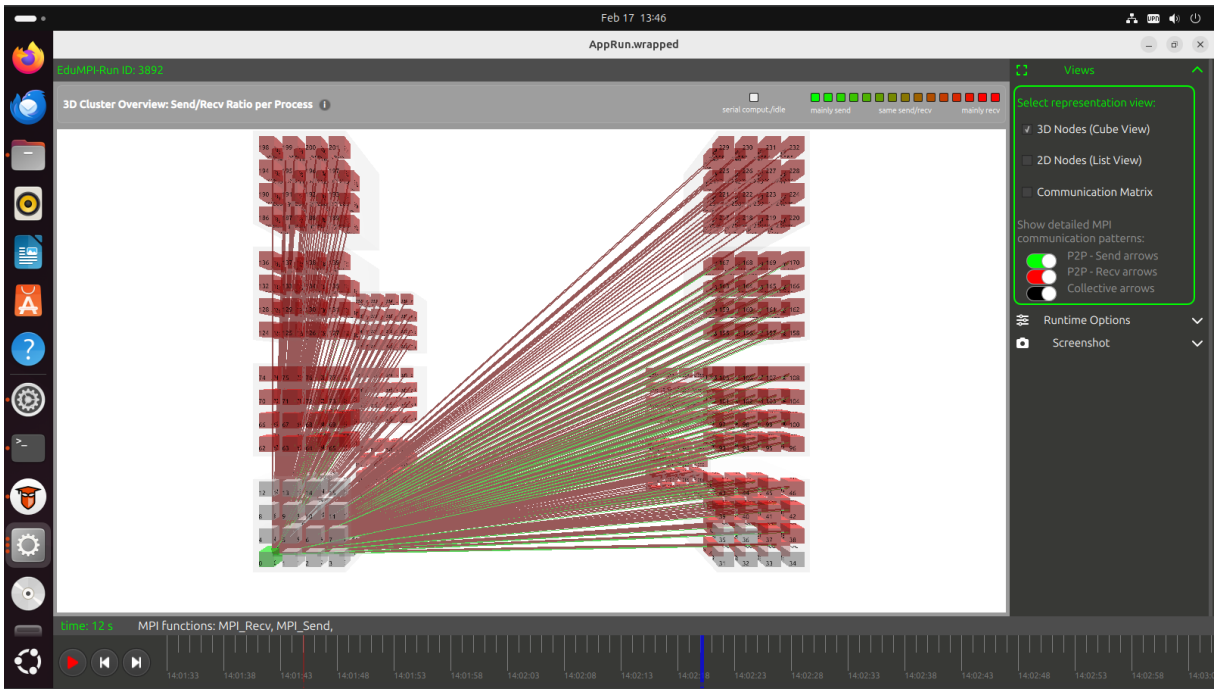


Figure 1: Heavy communication overhead (Red status) during row-by-row broadcast.

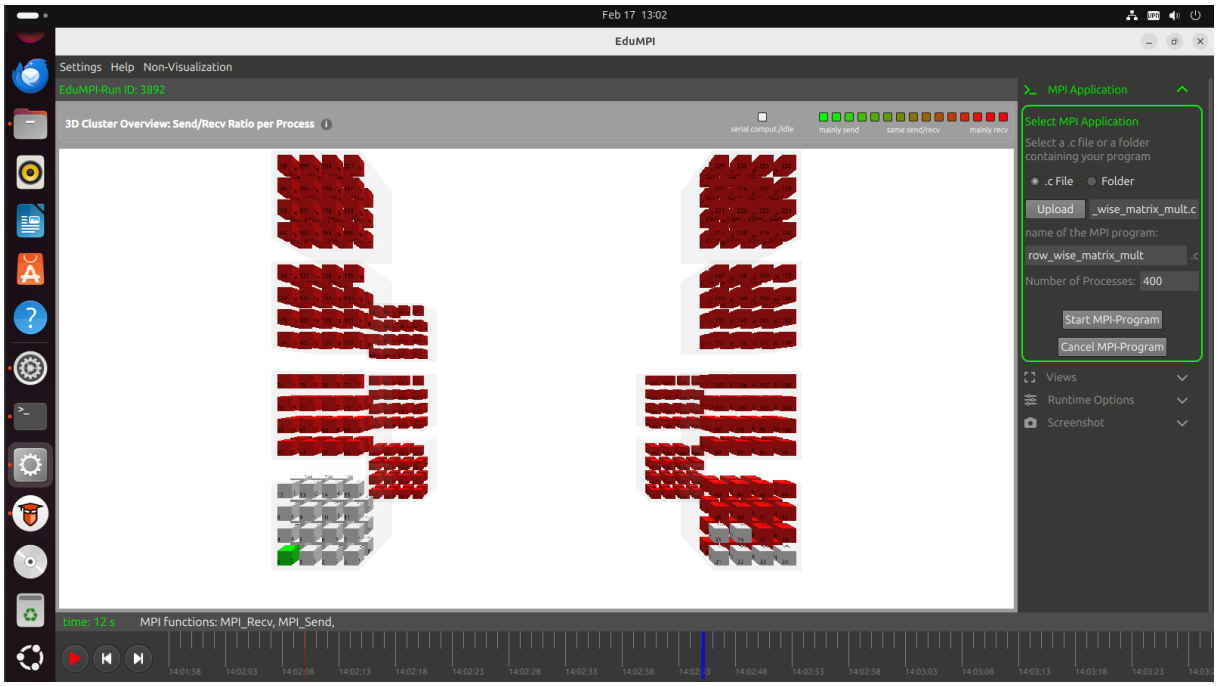


Figure 2: Processes waiting for data from Rank 0 (MPI_Recv/Send bottleneck).

Analysis of Optimized Implementation

The optimized implementation employs a **Geometric Decomposition** (1D Row Block) strategy. This approach statically divides the problem size, eliminating the overhead of the Master-Worker handshake and removing Rank 0 as a computational bottleneck.

Scalability and Parallel Initialization: A major improvement is the parallelization of data generation. Instead of Rank 0 generating the data, every rank calculates its own slice of Matrix A and a partial slice of Matrix B concurrently. By calculating global offsets (e.g. `my_offset`) and using the provided RNG, the initialization time scales as $O(N^2/P)$. This effectively removes the serial portion of the code that previously limited performance at high processor counts (128+ ranks).

Communication Efficiency: Inefficient row-by-row broadcast is replaced by `MPI_Allgatherv`. Each rank generates a portion of Matrix B and shares it with all others in a single collective operation. This maximizes bandwidth usage and minimizes latency. The final gathering of results uses `MPI_Gatherv`, which is necessary because the result is needed on Rank 0 to print the checksum; otherwise, the calculation completes fully distributed.

Memory Efficiency: Memory consumption is significantly reduced. Rank 0 no longer holds the full global state during computation. Matrices A and C are distributed, meaning each node only requires $1/P$ of the memory for these arrays. While Matrix B is still replicated on all nodes (a requirement for 1D decomposition), the overall memory footprint is much more scalable than the unoptimized version.

Comparison of Alternate Optimization Strategies

During the development of the optimized solution, two distinct parallel approaches were explored by the team. Both methods correctly implement 1D Row Block decomposition for matrices A and C, and both utilize cache-friendly $i-k-j$ loop layouts for the matrix multiplication itself. However, they drastically differ in how they handle the initialization and distribution of Matrix B.

Approach 1: Distributed Generation with Collective Communication (Author's Implementation) In this approach, every process computes only its assigned horizontal slice of Matrix B. Once generated locally, an `MPI_Allgatherv` operation is called to share all slices, allowing every process to assemble the full contiguous Matrix B in memory.

- **Initialization Compute Time:** Scales exceptionally well at $O(N^2/P)$. As cluster size grows, the time spent generating random numbers shrinks proportionally.
- **Communication Overhead:** Introduces the network overhead of `MPI_Allgatherv`, which scales with the total bytes of Matrix B.
- **Memory Footprint:** Requires the allocation of a temporary buffer (`local_B_slice`) in addition to the memory for the full assembled Matrix B.

Approach 2: Redundant Local Generation (Teammate's Implementation) This approach bypasses collective communication entirely. Rather than generating a slice and distributing it, *every single process* generates the full $N \times N$ Matrix B locally using the pseudo-random number generator, relying on the deterministic nature of the PRNG seed to ensure all nodes hold identical data.

- **Initialization Compute Time:** Performs better at smaller matrix sizes, remaining fixed at $O(N^2)$ for Matrix B regardless of process count. Every node computes the exact same N^2 pseudo-random numbers.
- **Communication Overhead:** Completely eliminates the `MPI_Allgatherv` operation. Zero network bandwidth is consumed to distribute Matrix B.

- **Memory Footprint:** Marginally more memory efficient as it skips the creation of the temporary `local_B_slice` buffer.

Conclusion on Strategies: While Approach 2 successfully circumvents network bottlenecks by eliminating `MPI_Allgatherv`, it introduces an $O(N^2)$ redundant calculation bottleneck on every node. For massive matrix dimensions, the CPU time required to locally generate hundreds of millions of pseudo-random numbers will completely overshadow the time saved by omitting network communication. Approach 1 represents the more robust, scalable parallel solution. Despite the required collective communication, the initialization workload is properly distributed, allowing the entire application (both generation and computation phases) to achieve near-linear speedup as node count increases.

EduMPI Visualization of Optimized Implementation

Balanced Computation: In contrast to the unoptimized version, Figure 4 shows majority **gray cubes**. This color indicates that processes are spending the majority of their time performing calculations (Matrix Multiplication) rather than waiting on the network. This is achieved by removing the serial initialization bottleneck; every rank generates its own data in parallel ($O(N^2/P)$) rather than waiting for Rank 0.

Collective Communication Pattern: Figure 3 shows the `MPI_Allgatherv` operation utilized in Approach 1. Instead of a single bottleneck (Master), we see a dense, structured network of communication lines where every process shares its generated slice of Matrix B with every other process simultaneously. This maximizes the utilization of the cluster's aggregate bandwidth and significantly reduces the total time to solution.

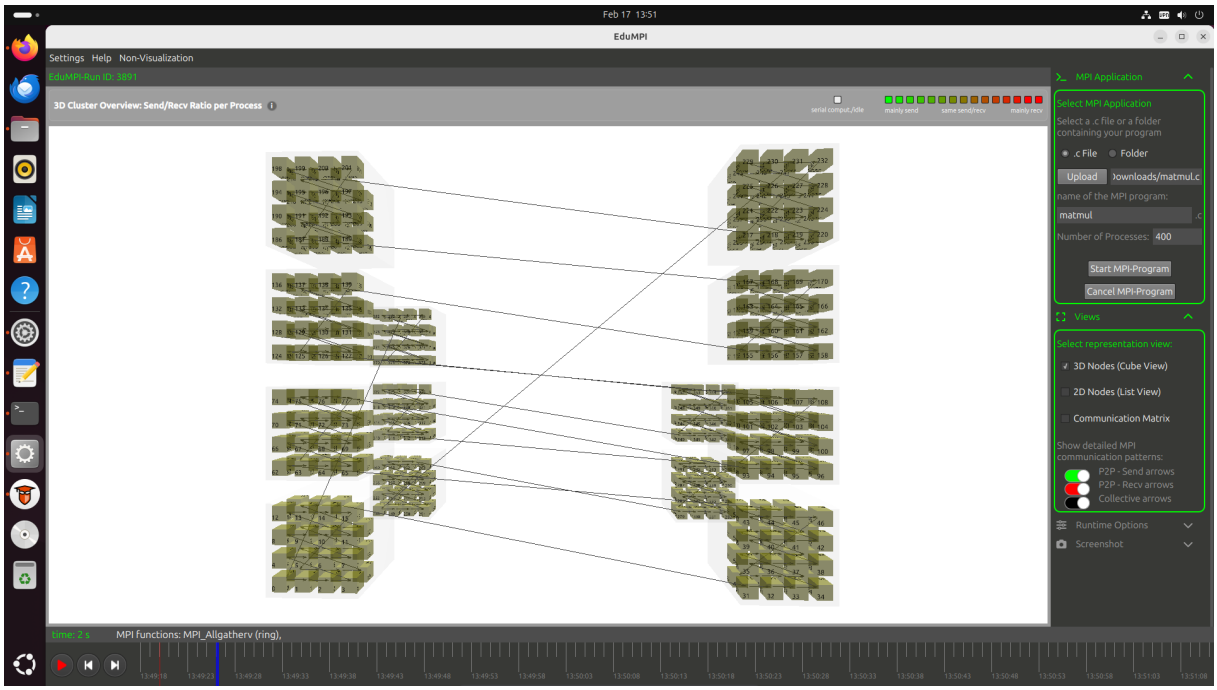


Figure 3: Visualization of the MPI_Allgatherv collective communication pattern.

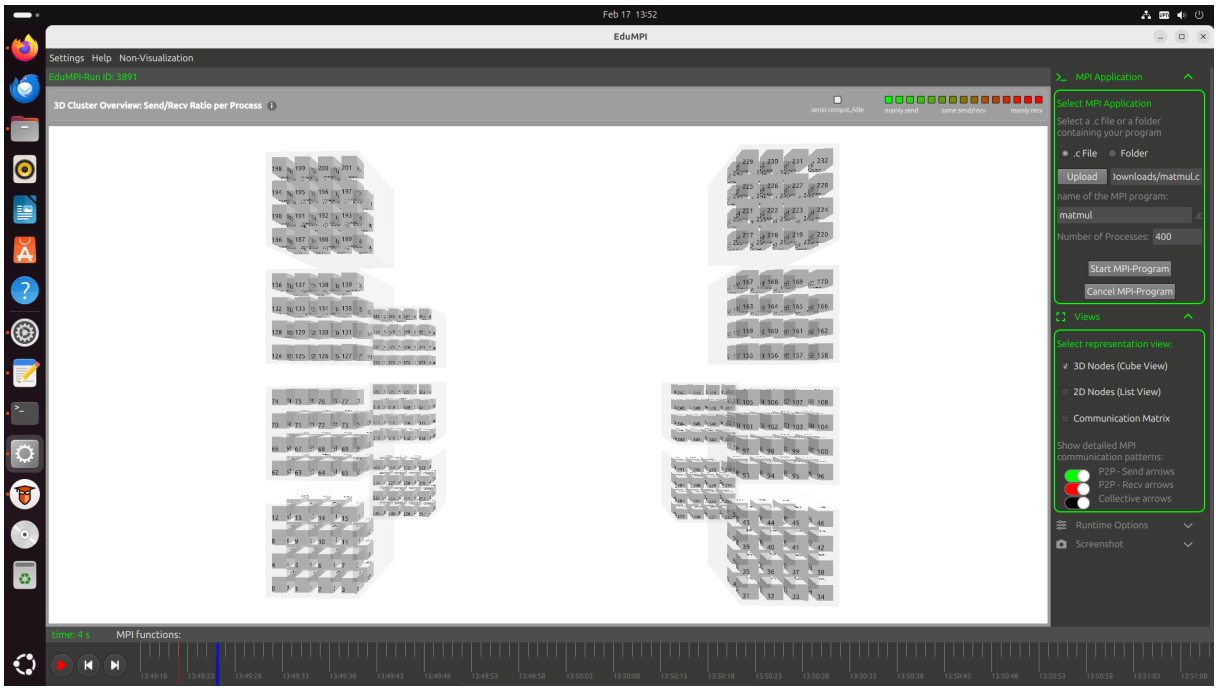


Figure 4: Cluster state showing idle communication for the majority of time.

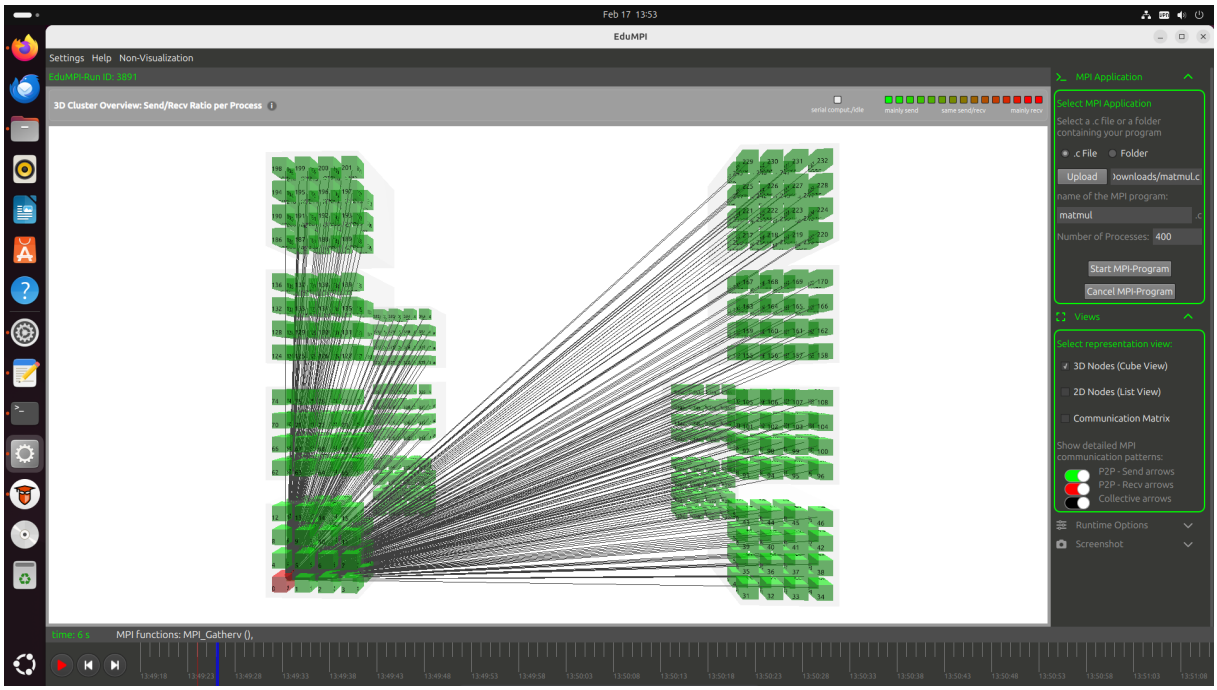


Figure 5: The final gathering of results uses MPI Gatherv, required to print the checksum.